CS 486/686
Neural Networks I: Neurons to CNNs

Yuntian Deng

Lecture 19

RN 21.1–21.3 · PM 7.5

Learning goals

- Describe the **simple mathematical model** of a neuron.
- Describe desirable properties of an **activation function**.
- Learn / identify a **perceptron** for a logical function.
- Explain why a perceptron **cannot represent XOR** — and how a hidden layer fixes it.
- Stack neurons into a **multilayer perceptron**.
- Understand **convolutional networks (CNNs)** for images.

Learning complex non-linear relationships

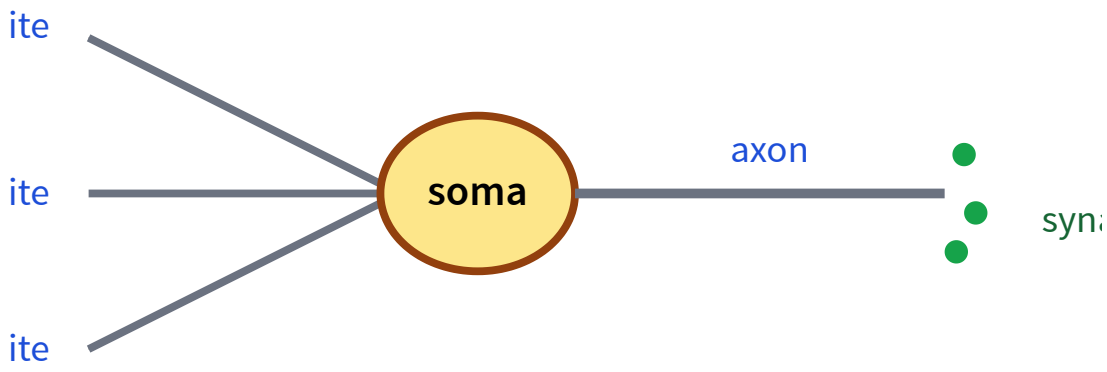
In vision, speech, translation, and beyond, the relationship between inputs and outputs is too complex to hand-program.

> We need a model that can:
>
> - Learn complex relationships.
> - Be trained efficiently on lots of data.
> - Generalize without overfitting.

Loose inspiration: the human brain — many simple components, densely connected.

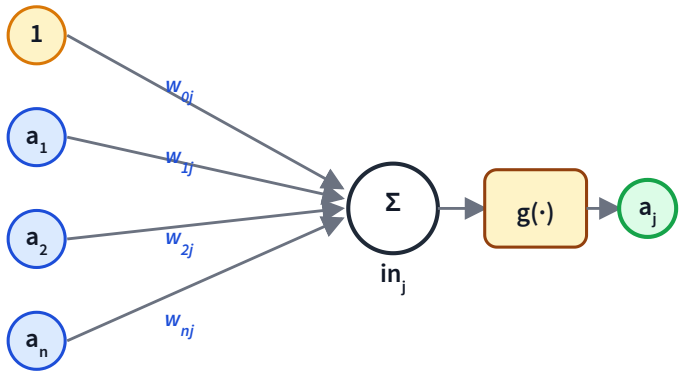
From brain neurons to math neurons

- **Dendrites** receive input signals.
- **Soma** processes the signals.
- **Axon** sends an output signal.
- **Synapses** connect neurons together.
- Neurons fire (or don't) based on the inputs.



The math model abstracts this into **weighted sum + non-linear activation**.

The math model (McCulloch & Pitts, 1943)



$$in_j = \sum_{i=0}^{n} w_{ij}\, a_i$$

$$a_j = g(in_j)$$

$a_0 = 1$ is the bias; w_{0j} acts as a threshold. g is a non-linear **activation function**.

Desirable properties of the activation function

Non-linear

Complex relationships are non-linear; stacking linear layers stays linear.

Mimics real neurons

Fires (≈ 1) when input is big; quiet (≈ 0) otherwise.

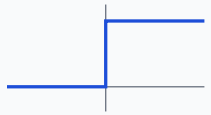
Differentiable

So we can train via gradient-based optimization (next lecture!).

Four common activation functions

Step

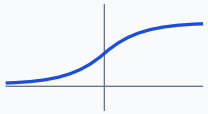
$$g(x) = 1 \text{ if } x > 0, \text{ else } 0.$$



Simple but *not differentiable*; rarely used in practice.

Sigmoid

$$g(x) = \frac{1}{1 + e^{-x}}$$



Smooth, but *vanishing-gradient* in saturation regions.

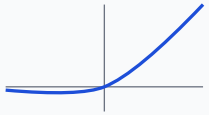
ReLU

$$g(x) = \max(0, x)$$

Fast, sparse; *dying-ReLU* when input stays negative.

GELU / SiLU

$$g(x) = x \cdot \sigma(x) \quad \text{(SiLU)}$$



Smooth, ReLU-like; the **modern default** in transformers/LLMs.

(Leaky ReLU adds a small negative slope to ReLU to keep the gradient alive.)

Networks: feedforward vs recurrent

Feedforward

Connections form a **DAG** (no loops). The network is just a function of its inputs.

Recurrent

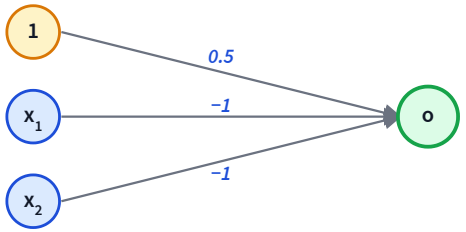
Outputs feed back as inputs to process **sequences** — we return to sequences (and their limits) in L21.

Perceptron

A **single-layer feedforward** network: inputs connect directly to outputs. Can represent AND, OR, NOT and many other logical functions.

Identify a perceptron

Q1. The perceptron below uses step activation with weights $w_0 = 0.5, w_1 = -1,$ $w_2 = -1$. What logical function does it compute?



A. $x_1 \wedge x_2$

B. $\neg(x_1 \wedge x_2)$

C. $x_1 \vee x_2$

D. $\neg(x_1 \vee x_2)$ (NOR)

D — NOR. The output is $g(0.5 - x_1 - x_2)$, which is 1 only when $0.5 - x_1 - x_2 > 0$, i.e. both x_1 and x_2 are 0.

Learn a perceptron: AND

Q2. Find weights (w_0, w_1, w_2) so the perceptron $o = g(w_0 + w_1 x_1 + w_2 x_2)$ computes $x_1 \wedge x_2$. The AND truth table outputs 1 *only* on $(1, 1)$.

A. $w_0 = -1, \;\; w_1 = 0.5, \;\; w_2 = 0.5$

B. $w_0 = 0.5, \;\; w_1 = -1, \;\; w_2 = 1$

C. $w_0 = 1.5, \;\; w_1 = -1, \;\; w_2 = -1$

D. $w_0 = -1.5, \;\; w_1 = 1, \;\; w_2 = 1$

D. Check: $-1.5 + x_1 + x_2 > 0$ iff $x_1 + x_2 > 1.5$, i.e. both equal 1. The other options either misclassify $(1, 1)$ or fire on $(0, 0)$.

More perceptron arithmetic

Step activation: $g(x) = 1$ if $x > 0$, else 0.

Q3. What does $h_1 = g(x_1 + x_2 - 0.5)$ compute?

A. $x_1 \vee x_2$ (OR)

B. $x_1 \wedge x_2$

C. $\neg(x_1 \vee x_2)$

D. $\neg(x_1 \wedge x_2)$

A — OR. Outputs 1 if at least one input is 1.

Q4. What does $h_2 = g(-x_1 - x_2 + 1.5)$ compute?

A. $x_1 \vee x_2$

B. $x_1 \wedge x_2$

C. $\neg(x_1 \vee x_2)$

D. $\neg(x_1 \wedge x_2)$ (NAND)

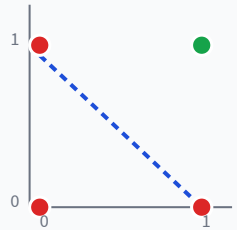
D — NAND. Outputs 0 only when both inputs are 1.

Remember h_1 and h_2 — we'll combine them to build XOR.

Why a perceptron can't represent XOR

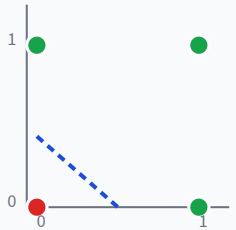
A perceptron is a **linear classifier**. Its decision boundary is a hyperplane in the input space.

AND



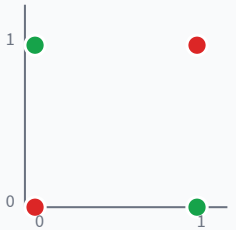
Separable

OR



Separable

XOR

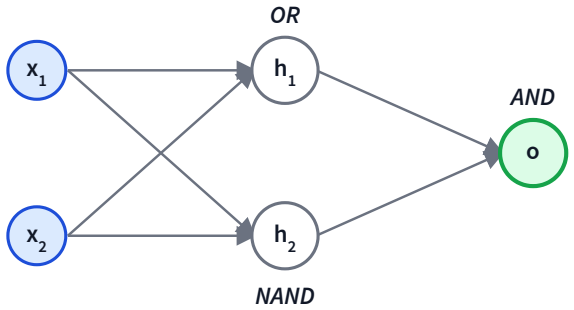


Not separable

Minsky & Papert (1969) showed this — and it triggered the first AI winter.

XOR as a 2-layer network

Rewrite XOR using gates a perceptron *can* handle: $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2) = h_1 \wedge h_2$.



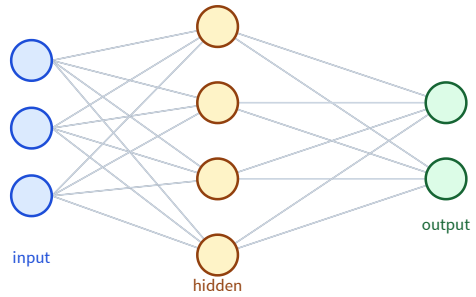
Truth-table check:

x_1	x_2	h_1 (OR)	h_2 (NAND)	$h_1 \wedge h_2$
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Matches XOR exactly. One hidden layer is enough.

Stacking neurons: the multilayer perceptron

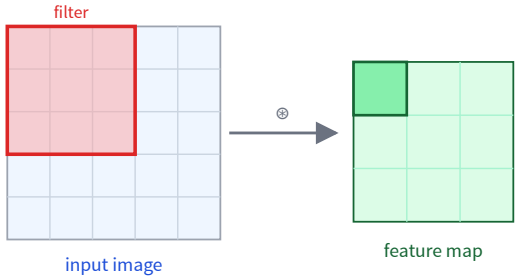
XOR needed one hidden layer. Stack layers of neurons and you get a **multilayer perceptron (MLP)** — a fully-connected feedforward network.



- Each hidden unit computes $g(\mathbf{w}^\top \mathbf{x} + b)$; layers compose into richer features.
- **Universal approximation:** a wide enough hidden layer can approximate any continuous function.
- Stack more hidden layers → a **deep** network. Output layer = softmax or linear (L17).

Convolutional networks: weight sharing for images

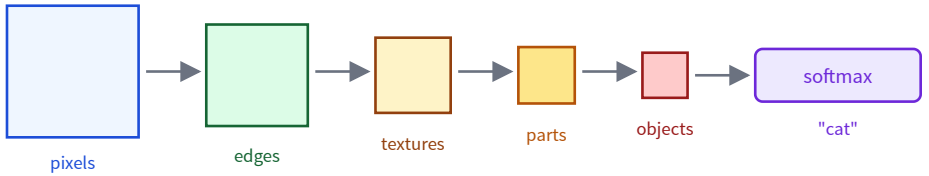
A fully-connected layer on a megapixel image needs billions of weights. A **CNN** instead slides one small **filter** over the image, reusing the same weights everywhere.



- **Local:** each output looks at a small patch (receptive field).
- **Weight sharing:** the same filter runs everywhere — far fewer parameters, and it detects a feature anywhere in the image.
- Many filters → many feature maps (edges, corners, textures …).

Pooling & the feature hierarchy

Alternate **convolution** (detect features) with **pooling** (shrink the map, keep the strongest response). Stacking these builds features from simple to complex.



- **Pooling** (e.g. 2×2 max) shrinks maps → small translations don't matter.
- Deeper layers see larger regions → edges → parts → whole objects.
- CNNs powered the deep-learning breakthrough in vision (ImageNet, 2012).

Learning goals (recap) — Next: training

✓ Describe the **math neuron** and activation functions.

✓ Learn / identify a **perceptron** for a logical function.

✓ Explain why a perceptron **cannot represent XOR** — and how a hidden layer fixes it.

✓ Stack neurons into a **multilayer perceptron**.

✓ Understand **CNNs**: convolution, weight sharing, pooling, feature hierarchy.

L20: how do we actually *train* these networks? — gradient descent + **backpropagation**.